

Assignment 2

Contents

Architectural Choices 1

Tokenization Choices 8

Constrained Decoding 15

Fine Tuning 19

Architectural Choices

Strategy

To extract the architectural specifications and design choices for each model, without needing to load the entire model, we leveraged the `AutoConfig.from_pretrained` feature from the Hugging Face transformers library. While this configuration object is primarily utilized by the framework to programmatically instantiate and initialize models, it also serves as a comprehensive, human-readable blueprint of the model's structural parameters.

A standard configuration for a decoder-only Large Language Model (LLM) is formatted as a JSON file, specifying fundamental hyper-parameters such as layer dimensions, tokenization constraints, and attention mechanisms:

```
{  
  "_name_or_path": "model_id",  
  "architectures": ["LlamaForCausalLM"],  
  "bos_token_id": 128000,  
  "eos_token_id": 128009,  
  "pad_token_id": 128004,  
  "hidden_act": "silu",  
  "hidden_size": 4096,  
  "initializer_range": 0.02,  
  "intermediate_size": 14336,  
  "max_position_embeddings": 8192,  
  "model_type": "llama",  
  "num_attention_heads": 32,  
  "num_key_value_heads": 8,  
  "num_hidden_layers": 32,  
  "rms_norm_eps": 1e-05,  
  "rope_theta": 500000.0,  
  "rope_scaling": null,  
  "tie_word_embeddings": false,  
  "torch_dtype": "bfloat16",  
  "transformers_version": "4.43.0",  
  "use_cache": true,  
  "vocab_size": 128256  
}
```

Depending on the paradigm of the underlying architecture, such as a Mixture-of-Experts (MoE) model, the configuration dynamically includes specialized attributes to define routing mechanisms, expert capacities, and active parameter counts.

To ensure empirical validity, we cross-referenced the extracted configuration attributes with each model's official technical report or whitepaper. In instances where discrepancies arose between the repository configuration files and the published documentation, further inspection was conducted. For example, the technical report for Llama-3.1-8B-Instruct specifies a vocabulary size of exactly 128,000, whereas the underlying config.json defines a vocab_size of 128,256. We attribute such minor variances to stylistic rounding conventions adopted in public documentation for readability, rather than actual structural deviations in the deployed model architecture.

To ensure maximum precision and reproducibility, architectural metrics were compiled through a combined programmatic and document-based extraction pipeline. The fields were discovered, parsed, and derived as follows:

- **Hidden Size:** The embedding dimensionality of each token vector across the transformer blocks. This property was extracted directly from the primary configuration file under the hidden size parameter.
- **Number of Layers:** The total depth of the network, representing the number of sequential transformer blocks. This was sourced straight from the hidden layers count parameter in the model configuration.

- **Attention Heads:** The number of query heads utilized in the multi-head attention mechanism. This value was located and extracted from the attention heads attribute in the configuration file.
- **Key/Value Heads:** The number of key and value heads, which can vary from the query head count depending on the attention variant used (such as Multi-Query Attention or Grouped-Query Attention). This property was extracted from the key/value heads configuration field, automatically defaulting to the number of query attention heads if the model uses standard Multi-Head Attention.
- **MLP Intermediate Size:** The dimensionality of the hidden layer within the feed-forward network block. This was extracted from the intermediate size field inside the configuration file.
- **Activation Function:** The specific non-linearity applied within the MLP block. This was pulled straight from the hidden activation property in the model configuration.
- **Normalization Scheme:** The type of normalization layer applied (such as RMSNorm or LayerNorm) and its structural placement (Pre-LayerNorm versus Post-LayerNorm). This information uniquely did not come from the model configuration as there is no standardized attribute for normalization type or layer placement, and attributes related to epsilon values are not uniform across model families. Therefore, this property was manually extracted from the official technical reports and source code repositories of each model and mapped via a local lookup configuration.
- **Positional Encoding Paradigm:** The method used to inject sequence order. This property was extracted by evaluating the presence of rotary embedding parameters, parsing specific attributes like the base frequency (theta), and checking nested positional mapping blocks inside the configuration file.
- **Context Length:** The maximum sequence token length the model can process during a single forward pass. This was pulled directly from the maximum position embeddings property in the model configuration.
- **Vocabulary Size:** The total number of unique tokens defined in the model's embedding matrix. This property was extracted directly from the vocabulary size configuration field.
- **Mixture-of-Experts (MoE) Architecture:** Specifications regarding whether the model utilizes sparse gating mechanisms, including the total number of experts, active experts per token, and routing type. This information was dynamically extracted by querying model-type definitions and locating nested expert intermediate and routed expert fields within the configuration file, defaulting to "Not Applicable" for dense architectures.
- **Head Dimension:** The vector dimensionality allocated to each individual attention head. This property was derived mathematically by dividing the Hidden Size by the number of Attention Heads, except in specialized architectures like DeepSeek-V3, where the distinct low-rank Multi-Head Latent Attention dimensions were parsed directly from their explicit configuration keys.
- **KV Group Size:** The ratio of query heads to key/value heads dictating the head sharing ratio in configurations like Grouped-Query Attention. This property was derived mathematically by dividing the number of Attention Heads by the number of Key/Value Heads.
- **Tied Word Embeddings:** A boolean indicator specifying whether the input embedding matrix and the output unembedding (projection) matrix share the exact same weights. This property was extracted straight from the tied embeddings boolean attribute in the configuration file.

- **Analytical Parameter Estimation:** Rather than relying solely on the approximate parameter counts indicated by model naming conventions (such as "8B"), the precise parameter count was mathematically derived using a script that aggregated the extracted structural variables (including number of layers, hidden size, intermediate MLP dimensions, vocabulary sizes, and expert distributions). This allowed us to calculate the exact theoretical parameter footprint and distinguish between total and active weights without the hardware overhead of loading the actual model matrices into memory.

Model-by-Model Architectural Summaries

1. Meta-Llama-3.1-8B-Instruct

The Llama-3.1-8B-Instruct architecture features an analytical parameter footprint of 8.03 billion parameters distributed across 32 layers with a uniform layer width (hidden size) of 4096 channels. The attention mechanism uses Grouped-Query Attention (GQA) with 32 query attention heads and 8 key/value heads, establishing an individual head dimension of 128 and a key/value group size of 4. The model implements an untied word embedding layout where the input embedding matrix and output unembedding projection matrix maintain separate weights over a vocabulary size of 128256. Its feed-forward blocks use a single non-linear activation type, a SiLU-activated SwiGLU gated MLP, with an intermediate layer dimension of 14336. Layer normalization is handled via a pre-norm RMSNorm scheme. To facilitate an extended context length of 131072 tokens, it implements Rotary Position Embeddings (RoPE) configured with a base frequency hyperparameter (theta) of 500000 and a specialized llama3x8.0 interpolation scaling factor.

2. Mistral-7B-Instruct-v0.3

The Mistral-7B-Instruct-v0.3 architecture is built upon 32 layers with a hidden layer width of 4096 channels, totaling an analytical count of 7.25 billion parameters. It utilizes a GQA configuration consisting of 32 query attention heads and 8 key/value heads, yielding a standard head dimension of 128 and a key/value group size of 4. It enforces untied word embeddings, mapping representations across a compact vocabulary layer of exactly 32768 tokens. The feed-forward network contains SwiGLU gated MLP layers running on a uniform SiLU activation function with an intermediate dimension of 14336. Normalization is structured in a pre-norm layout utilizing RMSNorm layers. The model supports a maximum position context length of 32768 tokens using a native RoPE design, which scales up its frequency hyperparameter to a base theta value of 1000000 without requiring additional active scaling modifiers.

3. Qwen2.5-7B-Instruct

The Qwen2.5-7B-Instruct model alters standard dimensions by deploying a depth of 28 layers and a narrower hidden layer width of 3584 channels, resulting in a derived parameter footprint of 7.62 billion weights. To maximize knowledge capacity within this frame, its feed-forward network drastically inflates the SwiGLU MLP intermediate dimension to 18944 channels using a consistent SiLU activation function. The attention block operates via a wide GQA layout featuring 28 query attention heads and 4 key/value heads, establishing a head dimension of 128 and an expanded key/value group size of 7. Word embeddings remain completely untied across a highly dense vocabulary of 152064 tokens. The model employs a pre-norm RMSNorm scheme across all blocks and supports a maximum context length of 32768 tokens, position-encoded via RoPE with an elevated base frequency theta of 1000000.

4. Allenai/OLMo-2-1124-7B-Instruct

The OLMo-2-1124-7B-Instruct model incorporates 7.3 billion parameters structured across 32 layers with a hidden width of 4096 channels. It deviates significantly from the other models by rejecting GQA in favor of classic Multi-Head Attention (MHA), maintaining an equal 32 query heads and 32 key/value heads for a key/value group size of 1 and a head dimension of 128. It also rejects the pre-norm standard, employing a post-norm / reordered RMSNorm configuration (applying normalization layers after the attention and MLP blocks) combined with an explicit QK-normalization step. The feed-forward blocks use a SiLU-activated SwiGLU MLP with a narrower intermediate dimension of 11008 channels. Word embeddings are untied over a vocabulary size of 100352 tokens. It natively handles an unscaled max context length of 4096 tokens, tracking positions via standard RoPE with a base theta of 500000.

5. Ibm-granite/granite-3.3-8b-instruct

The Granite-3.3-8b-instruct architecture scales out depth by distributing 8.17 billion parameters across 40 distinct layers with a hidden layer width of 4096 channels. Its feed-forward layer dimensions are bounded to an intermediate SwiGLU MLP size of 12800 channels running a uniform SiLU non-linearity. The attention component relies on a GQA setup consisting of 32 query attention heads and 8 key/value heads, translating to a head dimension of 128 and a key/value group size of 4. Crucially, Granite breaks modern open-weights conventions by enabling tied word embeddings, forcing the input embedding and final linear projection unembedding layers to share an identical weight matrix across a vocabulary of 49159 tokens. The model uses a pre-norm RMSNorm layer setup and expands its maximum position limit to 131072 tokens by implementing RoPE with an ultra-high base frequency theta hyperparameter of 10000000.

6. Deepseek-ai/DeepSeek-V3

DeepSeek-V3 represents a massive, sparse Mixture-of-Experts (MoE) architecture containing 671.03 billion total parameters, with 37.55 billion parameters active per token during any single forward pass. It is configured with 61 layers and an ultra-wide hidden layer width of 7168 channels. The architecture completely discards traditional multi-head attention layouts for Multi-Head Latent Attention (MLA), which compresses Key/Value states into a low-rank latent projection matrix to clear memory bottlenecks; it expresses a decomposed head structure where the Query-Key interaction dimensions equal 192 and the Value head dimension equals 128, working with a total of 128 attention heads and 128 key/value heads, creating a key/value group size of 1.

Its position encoding is decoupled: RoPE is applied to a separate, isolated 64-dimensional head slice using a base theta of 10000 and advanced YaRN x40 scaling to process a massive maximum context length of 163840 tokens. For its processing layers, the first 3 blocks remain standard dense SiLU SwiGLU MLPs (intermediate size of 18432), while the remaining 58 layers route tokens dynamically across 256 sparse routed experts and 1 heavily weighted shared expert. Each routed expert uses a SwiGLU dimension of 2048, and the model activates exactly 8 routed experts per token using a sigmoid gating scoring function. Word embeddings are untied over a vocabulary size of 129280 tokens, and the entire framework is normalized using a pre-norm RMSNorm configuration.

7. HuggingFaceTB/SmolLM2-1.7B-Instruct

The SmolLM2-1.7B-Instruct model represents a compact, mobile-optimized architecture with an analytical parameter allocation of 1.71 billion parameters structured across 24 layers. It features a narrower hidden layer width of 2048 channels. To achieve high arithmetic execution efficiency at this scale, it relies on strict Multi-Head Attention (MHA) utilizing an equal 32 query attention heads and 32 key/value heads, which yields a key/value group size of 1 and a scaled-down individual head dimension of 64 channels. The feed-forward network uses a SiLU-driven SwiGLU gated MLP with an intermediate dimension of 8192 channels. The model enables tied word embeddings to preserve precious memory capacity across a vocabulary size of 49152 tokens. It applies a pre-norm RMSNorm framework and processes a maximum position context length of 8192 tokens using a native RoPE setup with an explicit base frequency theta of 130000.

8. Microsoft/Phi-4-mini-instruct

The Phi-4-mini-instruct model is a highly compressed 3.84 billion parameter network running across 32 layers with a hidden width of 3072 channels. The feed-forward network maps to an intermediate SwiGLU MLP width of 8192 channels driven by a uniform SiLU activation. The attention engine uses GQA with 24 query attention heads and 8 key/value heads, setting an individual head dimension of 128 and a key/value group size of 3. The model implements tied word embeddings, meaning its embedding and unembedding layers share parameters across an extraordinarily large vocabulary matrix of 200064 tokens. It enforces a standard pre-norm RMSNorm framework. For spatial mapping, it relies on a partial RoPE strategy where a rotary factor hyperparameter of 0.75 applies the transformation to only a fraction of the channels, using a base theta of 10000 and dynamic longrope scaling to achieve a massive max position context of 131072 tokens.

9. Tiiuae/Falcon3-7B-Instruct

The Falcon3-7B-Instruct architecture utilizes roughly 7.46 billion parameters distributed across 28 layers with a hidden width of 3072 channels. It alters standard scaling ratios by implementing an unusually massive SwiGLU MLP intermediate dimension of 23040 channels under a uniform SiLU activation. Its attention layer also breaks convention by utilizing an inflated individual head dimension of 256 channels, driving a GQA block configured with 12 query attention heads and 4 key/value heads for a key/value group size of 3. Word embeddings are untied, mapping tokens over a vocabulary layer size of 131072. The model relies on a standard pre-norm RMSNorm framework and maps a maximum position context of 32768 tokens using an unscaled RoPE layout with an explicit base frequency theta of 1000042.

10. Dicta-IL/dictalm2.0-instruct

The dictalm2.0-instruct model is optimized with 7.25 billion parameters mapped across 32 layers with a hidden layer width of 4096 channels. Its feed-forward network tracks a standard 3.5x scaling ratio, resulting in an intermediate SwiGLU MLP dimension of 14336 channels running on a SiLU activation function. The attention block follows a standard GQA template composed of 32 query attention heads and 8 key/value heads, yielding an individual head dimension of 128 and a key/value group size of 4. It maintains an untied embedding layout, splitting the input and projection weights over a tailored vocabulary size of 33152 tokens. The model implements a standard pre-norm RMSNorm layer topology and resolves a maximum

sequence context of 32768 tokens using baseline, unscaled RoPE operations with a traditional base frequency theta hyperparameter of 10000.

Synthesis of Architectural Trends and Commonalities across the Sample

Grounding the findings directly in the nine analyzed models (Llama-3.1-8B, Mistral-7B, Qwen2.5-7B, OLMo-2-7B, Granite-3.3-8b, DeepSeek-V3, SmolLM2-1.7B, Phi-4-mini, and Dictalm2.0), there is a definitive architectural alignment around a modernized baseline configuration. Absolute consensus is achieved regarding core operational mechanics: every single model in this group has completely abandoned vanilla transformer feed-forward layers and traditional LayerNorm in favor of a SiLU-activated SwiGLU gated MLP and computationally efficient RMSNorm layers. Furthermore, pre-normalization serves as the overwhelming standard, being strictly enforced by eight out of the nine models to ensure smooth gradient flow through the residual stream during training. Position encoding has similarly reached full uniformity, with all models relying exclusively on Rotary Position Embeddings (RoPE), proving that absolute learned positions or relative bias matrices are obsolete in this cohort. Despite these shared foundations, a sharp ideological split exists regarding embedding weights and vocabulary design. While the majority of the models leave token embeddings untied, Microsoft's Phi-4-mini, IBM's Granite-3.3-8b, and HuggingFace's SmolLM2-1.7B buck this trend by implementing tied word embeddings to conserve parameter allocation. Tokenization strategies show zero convergence across the dataset, as vocabularies range from highly compact targets like Mistral-7B (32,768 tokens) and Dictalm2.0 (33,152 tokens) to expansive, multilingual schemas like Phi-4-mini (200,064 tokens) and Qwen2.5-7B (152,064 tokens).

Structural Deviations and Sizing Ratios in the Sample

A granular inspection of the mathematical dimensions across these specific architectures reveals strict scaling rules of thumb alongside drastic engineering deviations. The individual attention head dimension is rigorously anchored at 128 channels for seven of the models (Llama-3.1, Mistral, Qwen2.5, OLMo-2, Granite, Phi-4-mini, and Dictalm2.0), adjusting downward only for the mobile-focused SmolLM2-1.7B (64 channels) and scaling up uniquely for Falcon3-7B (256 channels). For feed-forward dimensions, the standard intermediate SwiGLU width scales precisely to a 3.5x ratio of the hidden size (14,336 intermediate channels versus a 4096 hidden size) in Llama-3.1-8B, Mistral-7B, and Dictalm2.0, or pinpoints a 2.66x (8/3) multiplier (8192 intermediate channels versus a 3072 or 2048 hidden size) in Phi-4-mini and SmolLM2-1.7B. Conversely, Falcon3-7B and Qwen2.5-7B aggressively expand their FFN layers to 23,040 and 18,944 channels respectively to maximize raw parameter density.

Two models fundamentally break the baseline template to solve distinct hardware scaling bottlenecks. OLMo-2-7B acts as a complete maverick by rejecting Grouped-Query Attention (GQA) for a standard 1:1 Multi-Head Attention layout, while swapping the pre-norm standard for a reordered, post-norm RMSNorm pipeline reinforced with explicit QK-normalization to preserve training stability at scale. DeepSeek-V3 stands out as an extreme architectural outlier, utilizing a massive 61-layer framework with a hidden size of 7168 that relies on Multi-Head Latent Attention (MLA) for low-rank joint KV cache compression, a decoupled 64-dimensional RoPE slice, and a sparse Mixture-of-Experts (MoE) FFN that selectively routes tokens across 256 routed experts and 1 shared expert. Finally, the configuration of the RoPE base frequency (theta) operates as an adjustable hyperparameter linked tightly to context limits; while standard 32k-context models like Mistral-7B, Qwen2.5-7B, Falcon3-7B, and Dictalm2.0 scale theta to roughly 1,000,000, the 131k long-context models either scale theta up to 500,000 with custom scaling

algorithms (Llama-3.1-8B) or inflate it natively to an ultra-high 10,000,000 base frequency (Granite-3.3-8b).

Empirical Blueprint Derived from the Studied Cohort

Synthesizing the core strengths of this specific cohort yields a highly competitive blueprint for an optimal next-generation 7B–8B parameter base model. Drawing from the consensus depth of Llama-3.1-8B, Mistral-7B, OLMo-2-7B, and Dictalm2.0, the recommended architecture utilizes 32 layers running a wide hidden dimensionality of 4096 channels. To optimize memory utilization during text generation without degrading context tracking, the attention engine should deploy Grouped-Query Attention modeled after the Llama-3.1, Mistral, Granite, and Dictalm2.0 configurations, matching 32 query heads to 8 KV heads (a KV group size of 4) at a standard head dimension of 128.

The feed-forward pipeline should mirror the highly stable 14,336 SwiGLU intermediate layer size (the 3.5x scaling ratio) running a SiLU activation within a pre-norm RMSNorm layout. Following the design choices of the majority (six out of nine models), word embeddings should remain untied to allow embedding and projection matrices to adapt independently. Finally, the configuration should capture a modern 131,072 token context window by adopting a high RoPE base frequency (theta) of 1,000,000 paired with active interpolation scaling, compressing sequences into a well-balanced vocabulary size of approximately 100,000 tokens—bridging the gap between the compact tokenizers of Mistral and Dicta and the highly dense layouts of Phi-4-mini and Qwen2.5.

Tokenization Choices

Strategy

To minimize computational overhead, tokenizers were loaded independently of the full model weights using the `AutoTokenizer.from_pretrained` utility. Accessing the tokenizer objects directly enabled the extraction of various structural metadata without heavy memory demands. This allowed us to pull key configurations as needed, using properties like the tokenizer's length to verify vocabulary capacity or inspecting backend classes to evaluate tokenization strategies, providing a flexible way to analyze each model's design choices.

Definition and Extraction Strategy of Tokenizer Properties

- **Tokenizer Type:** The core subword algorithm used to compress text sequences into numbers. This property was extracted programmatically by passing the loaded tokenizer object to a structural utility function that inspected its class configuration. Models running on a byte-level framework were classified as byte-level BPE, while models built on the alternative codebase were mapped directly as SentencePiece.
- **Vocab Size:** The total number of unique token representations available to the model. This value was extracted directly by querying the native length property of the instantiated tokenizer object inside the analysis script.
- **Special Tokens:** Non-textual filler or control tokens used by the large language model to track system instructions, conversational turns, or sequence boundaries. This inventory summary was extracted by iterating through the tokenizer's internal special tokens tracker, merging them with the added vocabulary dictionary to get an exact set count, and formatting the first few placeholder examples into a descriptive string.

- **Word Boundary Strategy:** The structural rules the tokenizer follows to differentiate separate words from concatenated word pieces. This was extracted by mapping the identified tokenizer family onto a programmatic conditional loader. If a model falls into the byte-level family, the script assigns a description detailing how leading spaces are captured as a safe unicode meta-byte while remaining characters attach directly. For SentencePiece families, the script records how leading spaces are represented by a distinct quarter-block meta-symbol that acts as a visible space marker.
- **Byte Fallback or Byte Level:** The algorithmic strategy used to process rare, unseen, or character-heavy inputs like complex emojis without triggering errors. This property was determined by checking the structural family type discovered during initialization. The analysis script assigned a full 256-byte alphabet designation if the core engine was natively byte-level, or a byte-fallback status using special bracketed byte representations if the engine utilized SentencePiece logic.
- **Average Tokens per English Word:** The relative subword compression efficiency when processing standard English text. Rather than running over trillion-token training datasets, which is fundamentally unfeasible, this property was extracted by processing a large, representative text corpus from a local data fixtures configuration. The script encoded the sample with special tokens disabled to count the raw token output, calculated the word count via a standard whitespace split, and divided the token count by the word count to derive a rounded efficiency baseline.
- **Average Tokens per Hebrew Word:** The relative subword compression efficiency when processing standard Hebrew text. Similar to the English benchmark, this property was extracted by encoding a dedicated Hebrew prose sample from the data fixtures file with special control symbols disabled. The total generated token count was divided by the number of whitespace-delimited word chunks to provide a directly comparable cross-lingual metric.
- **Tokenizer Backend:** The exact underlying software wrapper or implementation class executing the tokenization algorithm. This was extracted programmatically by accessing the name attribute of the tokenizer object's underlying Python class, exposing whether the script was interacting with the modern fast backend architecture or a specialized custom class instance.

Methodological Approach to Computing Average Token Metrics

To determine the cross-lingual efficiency of the tokenizers, calculating average tokens-per-word metrics over the models' original training datasets is mathematically and computationally unfeasible, given that modern pre-training data spans into the realm of trillions of tokens. Instead, this study implemented an optimized benchmarking strategy utilizing dedicated, large-scale text corpora fetched from the Hugging Face Hub. For the English evaluation baseline, text sequences were compiled dynamically from the test split of the Salesforce wikitext-103 corpus. For the Hebrew evaluation baseline, paragraphs were parsed and aggregated from the YanFren Hebrew Wikipedia dataset. Under this framework, these curated text streams are assumed to be linguistically representative of natural, real-world language distribution.

The benchmarking script parses these text corpora and calculates word counts using a basic whitespace separation strategy. While this method inevitably means that attached trailing punctuation is grouped and counted as a single unit with its adjacent word chunk, this setup aligns directly with how humans generally interpret word boundaries in standard reading applications. More importantly, this approach avoids the need for heavy, language-specific

syntactic parsers, ensuring that the word-counting baseline remains completely uniform and directly comparable across both English and Hebrew formatting conventions.

Model-By-Model Tokenization Strategies

1. Meta-Llama-3.1-8B-Instruct

The Meta-Llama-3.1-8B-Instruct tokenizer possesses a total vocabulary size of 128256 tokens. It utilizes a byte-level Byte-Pair Encoding (BPE) algorithm where a leading space before a word is encoded as the specific unicode meta-byte 'Ġ'. Tokens lacking this character indicate that they are part of an incomplete word and should continue the previous chunk; merging back to words is achieved by concatenating the subword byte strings and mapping them back to standard characters. The tokenizer includes 256 special or added control tokens, such as `<|begin_of_text|>` and `<|eot_id|>`, which do not represent standard text but instead encode systemic transitions or chat headers. Specifically, the `<|begin_of_text|>` token is used to initialize the model's state by marking the absolute beginning of an input stream, while the `<|eot_id|>` (End of Turn ID) token signals the precise end of a conversational turn, instructing the system to stop text generation and await user input. To handle rare inputs, it operates on a native byte-level full 256-byte fallback alphabet. Based on large text corpora evaluations, it records an average efficiency of 1.208 tokens per English word and 5.880 tokens per Hebrew word.

2. Mistral-7B-Instruct-v0.3

The Mistral-7B-Instruct-v0.3 tokenizer maintains a vocabulary of 32768 tokens and follows a SentencePiece configuration. It marks word boundaries by replacing a leading space with the quarter-block meta-symbol ' '. Subwords without this symbol are treated as incomplete pieces that continue the preceding token, merging back into words during detokenization by concatenating pieces and converting the block symbols back into spaces. The vocabulary contains 771 special or added tokens that encode control sequences and structural instructions, including `,`, `.`, and `!`. The token `<|start|>` explicitly marks the beginning of a new sentence or instruction block to establish the context entry point. The token `<|end|>` indicates the finality of a sentence or sequence, acting as a stop command for the text generator. The token `<|unk|>` serves as an unknown marker for out-of-vocabulary string fragments when a direct character mapping cannot be achieved. Unknown or unseen tokens are resolved via a byte fallback mechanism that converts rare characters into explicit hexadecimal byte indicators like `<0xNN>`. It computes an average of 1.347 tokens per English word and 6.042 tokens per Hebrew word.

3. Qwen/Qwen2.5-7B-Instruct

The Qwen2.5-7B-Instruct tokenizer scales up to an expansive vocabulary size of 151665 tokens. It utilizes a byte-level BPE algorithm with a full 256-byte alphabet. Word boundaries are denoted by a leading space encoded as the meta-byte 'Ġ', meaning any token without this character represents an incomplete continuation of the previous word. Detokenization merges pieces back by joining byte strings and mapping them to text. It contains 22 special or added tokens, including chat markers like `<|im_start|>`, `<|im_end|>`, and `<|endoftext|>`, to manage structural prompting sequences. The `<|im_start|>` and `<|im_end|>` tokens are functional chat template delimiters that explicitly frame the start and end boundaries of specific speaker roles, such as distinguishing system instructions, user queries, and assistant responses. The `<|endoftext|>` token marks the complete termination of a standalone document or text file. Its compression

efficiency evaluates to an average of 1.249 tokens per English word and a highly compact 2.401 tokens per Hebrew word due to its heavily extended, language-optimized vocabulary.

4. Allenai/OLMo-2-1124-7B-Instruct

The OLMo-2-1124-7B-Instruct tokenizer has a total capacity of 100278 tokens and implements a byte-level BPE design. Like other byte-level architectures, it uses the 'Ġ' meta-byte prefix to mark word boundaries, treating tokens without this prefix as incomplete continuations that merge back into words via byte string concatenation. It incorporates 22 special or added control tokens, such as <|endoftext|> and <|pad|>, to signify sequence endings and padding structures. The <|endoftext|> token acts as a sequence delimiter indicating the absolute end of a text block. The <|pad|> token serves as a structural placeholder used to fill shorter text sequences up to a uniform maximum length within a batch array, allowing for efficient parallel computation across hardware without altering semantic attention weights. It avoids unknown token errors using a full 256-byte base alphabet. Its benchmarking performance indicates an average execution of 1.209 tokens per English word and 5.881 tokens per Hebrew word.

5. Ibm-granite/granite-3.3-8b-instruct

The Granite-3.3-8b-instruct tokenizer is configured with a vocabulary size of 49159 tokens. Operating under a byte-level BPE framework with a native 256-byte alphabet, it relies on the 'Ġ' unicode character to dictate word boundaries; tokens missing this leading marker are treated as incomplete word pieces that combine during decoding via concatenated byte sequences. The vocabulary includes 26 special or added structural tokens, such as <|end_of_text|>, to manage boundary constraints. The <|end_of_text|> token serves as a global control flag that signals the exact conclusion of a generated response or text stream, prompting the inference engine to safely close the forward pass. It yields an average token overhead of 1.463 tokens per English word and 5.240 tokens per Hebrew word.

6. Deepseek-ai/DeepSeek-V3

The DeepSeek-V3 tokenizer contains a vocabulary size of 128815 tokens. It utilizes a byte-level BPE strategy with a standard 256-byte base alphabet. Word boundaries are indicated via the 'Ġ' meta-byte, where any text snippet lacking this character is processed as an incomplete word continuation that merges back into standard characters during decoding. The tokenizer manages 818 special or added tokens that represent systemic structural controls, such as < | begin of sentence | > and < | end of sentence | > enclosed in brackets. The < | begin of sentence | > token explicitly registers the starting boundary of a text generation or document context to clear the model's internal state, while the < | end of sentence | > token serves as a terminal boundary marker that denotes the successful completion of a full text sequence. Driven by its aggressive multi-lingual optimization, it scores an average of 1.204 tokens per English word and an efficient 2.783 tokens per Hebrew word.

7. HuggingFaceTB/SmolLM2-1.7B-Instruct

The SmolLM2-1.7B-Instruct tokenizer features a compact vocabulary capacity of 49152 tokens. It is built on a byte-level BPE architecture using a standard full 256-byte base alphabet. It indicates word boundaries using the 'Ġ' prefix, forcing tokens without this prefix to act as incomplete continuations that merge back to words during detokenization by re-assembling byte matrices. The model contains 17 special or added formatting tokens, such as <|im_start|>, <|im_end|>, and <|endoftext|>, to track conversational metadata. The <|im_start|> and

<|im_end|> tokens provide structured formatting by encapsulating the exact boundaries of conversational role entries within chat configurations, separating user inputs from assistant responses. The <|endoftext|> token functions as a global delimiter that marks the ultimate end of a complete document string or text prompt. It exhibits an average of 1.260 tokens per English word and 6.538 tokens per Hebrew word.

8. Microsoft/Phi-4-mini-instruct

The Phi-4-mini-instruct tokenizer deploys a massive vocabulary size of 200029 tokens. It tracks a byte-level BPE strategy anchored on a full 256-byte base alphabet. The model flags word boundaries using a leading space shifted to the 'Ġ' unicode character, meaning tokens lacking this character represent incomplete word parts that merge back during text decoding via sequential byte concatenation. It features a streamlined set of 12 special or added control tokens, such as <|endoftext|>. The <|endoftext|> token functions as a general terminal delimiter that signals the final boundary of a complete text segment or generated text layer. Benefiting from its massive vocabulary allocation, it demonstrates a highly efficient average overhead of 1.201 tokens per English word and 2.355 tokens per Hebrew word.

9. Tiiuae/Falcon3-7B-Instruct

The Falcon3-7B-Instruct tokenizer operates with a total vocabulary profile of 131072 tokens. It applies a byte-level BPE mechanism utilizing a full 256-byte alphabet fallback structure. Word boundaries are established using the 'Ġ' character prefix, where all tokens omitting this character represent incomplete continuation blocks that combine back into complete words via concatenated byte string reconstruction. The inventory contains a large block of 2024 special or added tokens, including <|endoftext|> and <|pad|>, to enforce structural text alignments. The <|endoftext|> token marks the final boundary of a text stream or text generation. The <|pad|> token operates as a neutral filler sequence used to pad varying text inputs to an identical token length, enabling synchronous hardware acceleration across batched inputs. It maps to an average text overhead of 1.370 tokens per English word and 5.241 tokens per Hebrew word.

10. Dicta-IL/dictalm2.0-instruct

The dictalm2.0-instruct tokenizer comprises 33152 tokens and leverages the SentencePiece framework. Word boundary identification mimics the SentencePiece standard, using the quarter-block symbol '▯' to flag leading spaces. Any subword fragment that does not contain this block symbol is handled as an incomplete word chunk, merging back into text during decoding by linking sequential subwords and translating the blocks back to standard spaces. It includes just 3 special tokens, , , and , to handle sequence limits. The and tokens define the absolute starting point and terminal ending point of an input or generated text string to manage sequence lengths during decoding. The token acts as a placeholder to represent unknown, unmappable characters that fall outside the main vocabulary matrix. Unseen or raw inputs trigger a byte fallback sequence that converts unknown text into hexadecimal indicators like <0xNN>. Optimized specifically for Hebrew syntax, it evaluates to an average of 1.356 tokens per English word and an exceptionally low 2.723 tokens per Hebrew word.

Understanding the Average Tokens-per-Word Metric

The average tokens-per-word metric represents the compression efficiency of a language model's tokenizer when translating human text into machine-readable numerical IDs. A

theoretical ratio of 1.0 would mean that every single whitespace-separated word maps perfectly to exactly one token. In practice, tokenizers break down rare, complex, or morphologically rich words into smaller subwords or individual characters. Analyzing this metric across the ten profiled models reveals a profound disparity between English and Hebrew, shedding light on how vocabulary size, language architecture, and script optimization affect model behavior.

The Clear Disparity: English vs. Hebrew Efficiency

Across all ten models, the token-per-word ratio for English remains exceptionally stable, hovering tightly between 1.201 (Phi-4-mini) and 1.463 (Granite-3.3). This indicates that the vocabulary allocation for English is highly optimized; the vast majority of common English words are recognized as single, complete tokens, with only occasional splits occurring for complex prefixes, suffixes, or typos.

In stark contrast, Hebrew text experiences a massive efficiency penalty, with ratios spanning from 2.355 (Phi-4-mini) to as high as 6.538 (SmolLM2). This means that a Hebrew sentence requires anywhere from two to nearly seven times as many tokens as an English sentence of equivalent word length.

Linguistically, this occurs because Hebrew is a highly synthetic and agglutinative language. Prepositions, articles, and pronouns (such as "and," "the," "in," or "that") are attached directly to the front of a root word as prefixes, while possessives are attached as suffixes. This architectural property creates an exponentially larger pool of unique word variations than English. If a tokenizer is not explicitly exposed to these diverse Hebrew combinations during training, it fails to find whole-word matches and is forced to fragment the Hebrew word into basic subwords, standalone letters, or raw byte blocks.

The Role of Vocabulary Size and Targeted Training

The data demonstrates a clear correlation between a model's total vocabulary size and its tokenization efficiency in non-Western languages.

Models with heavily expanded or multilingual vocabularies achieve significantly better compression for Hebrew text. For example, Microsoft's Phi-4-mini-instruct deploys a massive vocabulary of 200,029 tokens and achieves the lowest Hebrew ratio at 2.355 tokens per word. Similarly, Qwen2.5-7B-Instruct uses a 151,665 token vocabulary to achieve an efficient 2.401 ratio, and DeepSeek-V3 utilizes 128,815 tokens to secure a 2.783 ratio. By expanding their vocabulary boundaries, these models can dedicate specific token slots to multi-character non-Latin text strings, allowing them to capture larger Hebrew subwords.

Another successful path is targeted language optimization. The dictalm2.0-instruct model possesses a relatively small overall vocabulary of 33,152 tokens, yet it manages a highly competitive Hebrew ratio of 2.723. This occurs because its training pipeline explicitly prioritized Hebrew text distribution, filling its compact vocabulary layer with high-frequency Hebrew combinations rather than crowding it with multi-lingual clutter.

Conversely, models with smaller generic vocabularies or architectures centered primarily on Western text distribution suffer from a severe token inflation. For example, SmolLM2-1.7B-Instruct (vocabulary of 49,152) scores a massive 6.538 Hebrew ratio, and Mistral-7B-Instruct-v0.3 (vocabulary of 32,768) scores 6.042. Interestingly, even Meta-Llama-3.1-8B-Instruct, despite its expanded 128,256 token vocabulary, registers a high Hebrew ratio of 5.880, indicating that its vocabulary table remains heavily weighted toward Latin-script optimization.

Downstream Technical, Operational, and Financial Implications

The structural variance exposed by the tokens-per-word numbers is not merely a theoretical quirk; it directly dictates the downstream performance, cost, and speed of using these models in real-world applications.

First, token inflation causes severe Context Window Shrinkage. Large Language Models possess strict context limits bounded by total tokens, not words. If a model like Llama-3.1 has a maximum context window of 131,072 tokens, it can process roughly 108,000 English words in a single forward pass. However, due to its high Hebrew ratio, that exact same context window can only accommodate around 22,000 Hebrew words before running out of memory. This drastically reduces the model's effective capacity to read long documents, summarize books, or maintain extensive chat histories when interacting in Hebrew.

Second, this disparity introduces a localized Financial Premium or Token Tax. Commercial API vendors and cloud infrastructure systems charge consumers strictly on a per-token basis for both input prompts and output generation. Because an unoptimized tokenizer breaks Hebrew down into five times as many pieces as English, running a business application or processing documents in Hebrew on models like Llama-3.1 or Mistral becomes roughly five times more expensive than processing an identical amount of semantic information in English.

Third, token fragmentation increases Generation Latency. Large language models generate text auto-regressively, executing a full forward pass through the entire neural network to produce one token at a time. Because a Hebrew concept requires significantly more individual tokens to express than its English counterpart, a model will take far longer wall-clock time to generate a sentence in Hebrew, increasing waiting times for end users.

Finally, extreme fragmentation can degrade overall Model Reasoning and Intelligence. When a tokenizer fragments a word down to single characters or raw byte fallback blocks (such as SmolLM2 or Mistral breaking text into individual bytes), the model's transformer layers are forced to spend their internal attention weights reconstructing basic spelling patterns and gluing character fragments together. This leaves less computational attention bandwidth available for the model to parse deep semantic associations, logical relationships, and high-level reasoning tasks across the broader context.

Analysis of Tokenization Divergence

Target Text: Supercalifragilisticexpialidocious

This iconic 34-letter non-standard compound word serves as an ideal edge-case to highlight the distinct segmentation rules, vocabulary constraints, and structural frameworks across different large language models. The analysis isolates three highly distinctive tokenization behaviors from the sample set:

1. Meta-Llama-3.1-8B-Instruct The model splits the text into the following fragments: Sup, erc, al, if, rag, il, istic, exp, ial, id, ocious. This pattern represents a greedy byte-level BPE segmentation that lacks the specific whole-word subword token "Super" or prioritizes a split starting with "Sup" and "erc". However, it possesses highly consolidated morphemes further down the sequence, recognizing complete blocks like "istic" and "ocious". Out of the remaining seven tokenizers, 2 models (Qwen/Qwen2.5-7B-Instruct and allenai/OLMo-2-1124-7B-Instruct) agree with this exact sequence completely,

showing a shared algorithmic lineage or highly correlated training corpus structures for processing long, rare strings.

2. **Mistral-7B-Instruct-v0.3** The model splits the text into the following fragments: ' Super', cal, if, rag, il, ist, ice, xp, ial, id, oc, ious. This pattern reveals two unique structural properties. First, it utilizes the SentencePiece framework, which manifests explicitly here through a leading space meta-symbol before "Super" at the start of the sequence. Second, its vocabulary splits subwords aggressively into smaller phonetic increments compared to Llama, breaking "istic" into "ist" and "ice", "exp" into "xp", and "ocious" into "oc" and "ious". Out of the remaining seven tokenizers, 1 model (dicta-il/dictalm2.0-instruct) agrees with this sequence completely. This alignment occurs because DictaLM is explicitly built on top of the SentencePiece framework, inheriting identical character normalization rules and base whitespace flags.
3. **Microsoft/Phi-4-mini-instruct** The model splits the text into the following fragments: Super, cal, if, rag, il, istic, exp, ial, id, ocious. This represents an optimized byte-level BPE pattern that maintains clean, whole semantic morphemes. Unlike Llama, its vocabulary contains the full subword "Super". Unlike Mistral, it preserves the large subword chunks "istic", "exp", and "ocious" as single units without fragmentation, indicating a highly efficient Latin-script vocabulary alignment for complex words. Out of the remaining seven tokenizers, 0 models agree with this precise sequence completely. While other remaining models also start with the clean subword "Super", they individualize further down the chain with unique vocabulary combinations (for example, SmolLM2 splits "exp" into isolated characters "x" and "p", while Falcon3 uses "istice" and "xp").

Constrained Decoding

Strategy for Identifying the Hebrew Vocabulary

To build a language-constrained text generation framework, the pipeline relies on a rigorous, multi-tier auditing strategy to scan a language model's complete vocabulary table. Instead of relying on static word lists, the strategy analyzes every individual token identifier sequentially to evaluate its structural and linguistic capacity to participate in valid Hebrew text construction. This process reduces the active vocabulary space down to an isolated, language-restricted subset.

The identification strategy is divided into four primary evaluation phases:

1. Foreign Script Rejection (Cross-Lingual Disqualification)

The initial phase acts as a strict filtering layer to prevent cross-lingual drift and code-switching during generation. Every token is decoded into its base text representation and evaluated against a comprehensive structural pattern containing foreign scripts. If a token contains any characters belonging to an external alphabet, such as Latin (English), Cyrillic, Arabic, Greek, or East Asian CJK scripts, it is immediately disqualified and discarded. This ensures that the model cannot slip into generating multi-lingual phrasing or foreign words.

2. Native Hebrew Script Validation

Tokens that survive the foreign script filter are inspected to confirm they contain valid language data. The strategy scans the decoded text strings for any characters falling inside the native Hebrew Unicode blocks. This includes:

- The primary Hebrew Unicode range (extending from U+0590 to U+05FF), which captures standard letters, final forms, vowels, and biblical points.
- The Hebrew alphabetical presentation forms range (extending from U+FB1D to U+FB4F), which accounts for specialized stylistic character combinations.

If an explicit Hebrew character is detected, the token is verified as linguistically valid and accepted into the approved set.

3. Language-Neutral Structural Retention

A complete sentence cannot be formed using letters alone; it requires numbers, formatting, and spacing. To ensure the model can generate cohesive text layout structures, the strategy implements a typographic fallback loop. Non-letter tokens are explicitly approved and retained if they meet either of the following criteria:

- The token consists entirely of characters within an authorized layout set, which includes numerical digits (0–9), standard formatting elements (tabs, newlines, standard spaces), primary punctuation marks, and localized symbols like the Israeli Shekel sign (₪).
- The token matches a universal Unicode typographic designation for punctuation, numbers, or structural whitespace separators.

By treating these layout symbols as shared resources across scripts, the model retains the ability to construct well-formatted text layers.

4. UTF-8 Byte Fragment Reconstruction

Because Hebrew letters are multi-byte characters when encoded in UTF-8, vocabulary tables frequently contain partial subword fragments or isolated binary slices rather than whole characters. If the strategy evaluated these segments purely as text strings, they would manifest as unmappable characters and be discarded. This would break the model's generation capabilities whenever it needs to assemble an uncommon word out of smaller building blocks.

To solve this, the strategy audits the underlying binary signatures of these textless fragments directly against the mathematical boundaries of the Hebrew UTF-8 encoding standard. In a standard UTF-8 stream, Hebrew characters are structured as two-byte sequences adhering to specific rules:

- The lead byte must be a hexadecimal value of either 0xD6 or 0xD7.
- The subsequent continuation byte must fall within the hexadecimal range of 0x80 to 0xBF inclusive.

If a raw byte token consists exclusively of these specific leading or continuation signatures, it is classified as a valid Hebrew participating element and kept. This low-level safety pathway allows the generation engine to smoothly concatenate raw byte blocks to construct rare words when a pre-assembled whole subword token is absent from the core vocabulary.

Constrained Decoding Strategy

The technical implementation of the language-constrained text generation pipeline is achieved dynamically at the inference layer using the Hugging Face LogitsProcessor interface. This methodology acts as a runtime filter, intercepting and adjusting token prediction score arrays at each progressive autoregressive step without altering the underlying neural network weights or requiring model fine-tuning. The process initializes by instantiating a vocabulary-wide boolean tensor mask matching the exact dimensions of the model's token matrix, where indices targeting safe text tokens are left unblocked while all remaining unallowed indices are flagged for suppression. At runtime initialization, the decoding script programmatically intercepts this mask to force open the slots for the model's native structural markers, specifically the End-of-Sequence token and the batch formatting index. This structural protection isolates the linguistic properties of the output text while ensuring the model maintains its fundamental capacity to process parallel generation batches and terminate safely upon hitting a normal text boundary. During each forward pass of the generation loop, the model computes a vector of raw probability scores, known as logits, for every potential item in the vocabulary table. The custom processor intercepts this tensor before the softmax or sampling layer is applied, running a vector-wide fill operation that overrides the score of every unauthorized or blocked index to negative infinity. This mathematical adjustment drops the selection probability for those tokens to absolute zero, forcing the downstream sampling or greedy search loops to choose exclusively from the approved vocabulary subset. Additionally, the processor incorporates an automatic resizing block to handle optimized vocabulary environments, such as Qwen2.5, where the raw output logit matrix is padded beyond the standard tokenizer array boundaries to optimize parallel computation hardware. The implementation dynamically tracks this logit width disparity, extending the mask on the fly and flagging all extra tail slots as blocked to prevent runtime out-of-bounds tensor crashes.

When evaluating the models under the baseline configuration (which can be found `constrained_decoding.csv`) where the vocabulary mask is active but the input query remains entirely in English, neither model produced any meaningful Hebrew text, collapsing instead into repetitive sequences of code syntax, mathematical symbols, markdown punctuation, and incremental number strings. This complete breakdown supports the hypothesis that the underlying models were simply not trained to transition directly from an English context into Hebrew tokens without explicit instructional adaptation. In a transformer architecture, an English prompt primes the model's internal attention layers to configure their hidden activation states to predict high-probability English continuations. When the logit-masking engine abruptly drops the scores of all Latin characters to negative infinity, the model cannot dynamically realign its historical attention weights or instantly force a sudden transition into Hebrew vocabulary tokens because the transitional probability of jumping from an English question into a native Hebrew word embedding is practically zero within the model's pre-trained weight matrix. Forced to select from a restricted vocabulary space, the model searches for the next highest valid probability remaining in its array. Because language-neutral tokens, such as digits, periods, comment slashes, and spaces, frequently co-occur with English text in standard training datasets, these universal punctuation tokens inherit the higher relative probabilities under the baseline test. Once the model samples these initial formatting characters, its own localized historical context traps it in a degenerative attention loop where the system becomes stuck repeating punctuation and numbers, proving that a mathematical logit mask is entirely passive and cannot force a model to generate coherent semantic text if the internal attention

states remain fixed in an alternate language domain. Under these conditions neither model produced meaningful Hebrew text.

The behavior of the generation loop changes completely upon switching to the context-injected configuration tracked in `decoding_outputs_system_prompt.csv`. To verify that the underlying logit processor was dynamically functioning as intended and genuinely allowing Hebrew tokens to pass through the masking layer rather than completely blocking them, a follow-up experiment was performed using an explicit system-level guiding directive. By prepending this instruction forcing the model to always respond in fluent Hebrew, the underlying input text context is fundamentally shifted before token selection begins. This instructional prefix acts as an internal directional anchor, pre-activating the transformer's translation pathways and Hebrew subword networks. When the forward pass executes, the top candidate tokens output by the model's logit layers are already valid Hebrew word pieces. Because the structural vocabulary mask cleanly permits these characters while filtering out any Latin characters the model might otherwise use to code-switch, the system prompt and the logit mask work in harmony, verifying that a proper constrained decoding strategy was successfully implemented, even though without the system prompt Hebrew output was not seen due to the default contextual token probabilities.

While the system instruction successfully unlocked Hebrew text generation in both models, a sharp disparity in semantic capability and contextual relevance was observed between the two systems, directly highlighting the impact of vocabulary size on model reasoning. Qwen2.5-7B-Instruct consistently delivered highly coherent, grammatically fluid, and contextually accurate responses across all ten prompts, naturally mapping incoming English concepts into fluent Hebrew grammar and producing direct, accurate answers. This success is tied directly to Qwen's large, language-optimized vocabulary matrix of 151,665 tokens, which contains thousands of whole-word Hebrew embeddings, allowing its hidden layers to immediately process complex semantics without exhausting their parameter capacity on multi-token character assembly. Conversely, Mistral-7B-Instruct-v0.3 suffered a near-complete breakdown in semantic meaning, producing chaotic sentences or word salads that failed to answer the underlying prompt. This failure stems directly from its compact vocabulary size of 32,768 tokens and its lack of high-level Hebrew entries, forcing Mistral to construct basic words piece-by-piece using individual single-byte fallback tokens. Because the model must expend its attention bandwidth simply trying to stitch byte fragments together to form valid word surfaces, it lacks the remaining computational capacity to maintain overall logical sentence structure, contextual relevance, or accurate reasoning.

Ultimately, this experiment demonstrates that modern language models are heavily biased toward the scripts and formatting patterns that dominate their baseline training data. Hard vocabulary constraints applied at the logit layer are highly effective formatting tools, but they cannot act as a substitute for native training exposure. If a model lacks a vocabulary structure natively optimized for a specific script, forcing it to generate text within that script will result in a severe penalty to its overall intelligence and operational coherence. We think that even though we allowed only Hebrew characters, the model was not trained this way, and that is exactly why punctuation or numbers got higher probabilities under the baseline test, confirming that masking logits can successfully block unauthorized text, but genuine cross-lingual capability requires deep semantic grounding within the model's core vocabulary layer.

Fine Tuning

Data Curation

To establish a robust training dataset without risking evaluation contamination or semantic stagnation, a scalable synthetic instruction generation pipeline was engineered using the GPT-5.4 Mini language model. The bootstrapping process was initialized by feeding the model a curated set of handwritten seed query-response pairs along with ten explicit thematic categories designed to maximize operational and conceptual diversity: everyday factual questions, short how-to instructions, simple science explanations, writing a short polite message, giving advice or tips, summarizing a common story, comparisons between two concepts, definitions of common terms, math word problems, and opinions and suggestions.

To enforce strict generalization boundaries and ensure the integrity of the experiment, GPT-5.4 Mini was explicitly prompted with the evaluation dataset and instructed to avoid creating queries similar to the test distribution. Newly created instructions were dynamically and programmatically cross-referenced against the entire held-out evaluation dataset to ensure absolute structural isolation and prevent any trace of downstream data leakage. Concurrently, an automated intra-loop deduplication pass was deployed, validating newly synthesized queries against one another to eliminate identical phrasing and redundant semantic overlaps. Following each successful instruction verification round, GPT-5.4 Mini was prompted to generate a corresponding, contextually accurate response in fluent Hebrew. This closed-loop creation, validation, and deduplication cycle was executed iteratively until the target baseline data size was fully satisfied, resulting in a clean, instruction-rich, and completely leak-free parallel corpus optimized for cross-lingual behavioral fine-tuning.

Fine Tuning with LoRA

To optimize the language model for script purity, semantic fidelity, and hallucination-free generation, a rigorous series of empirical fine-tuning iterations was conducted. The underlying Low-Rank Adaptation (LoRA) architecture was configured to maximize the model's expressive capacity by targeting all weight matrices within both the self-attention blocks (q_proj, k_proj, v_proj, o_proj) and the multi-layer perceptron (MLP) layers (gate_proj, up_proj, down_proj). The adaptation parameters were anchored at a foundational rank of $r = 16$ and an alpha scaling coefficient of $\alpha = 32$. This geometric scaling structurally constrained the updated weight matrices to follow a linear delta formula where α divided by r multiplies the low-rank trainable tensors, effectively doubling their mathematical scaling weight. To ensure the optimization loop focused strictly on directional text completion rather than prompt memorization, standard Supervised Fine-Tuning (SFT) masking was applied, setting the target labels of the user instructions to -100 and computing the cross-entropy loss exclusively on the Hebrew assistant responses. LoRA dropout was fixed at a conservative regularizer of 0.01.

Initial exploratory models were trained in a low-data regime using a small parallel corpus of 300 synthetic pairs. Under this constraint, the parameter boundaries completely failed to generalize. Regardless of the optimization combinations explored, including sweeping varying learning rates, learning rate schedulers, and physical batch sizes, the model could not achieve stable convergence or produce semantically coherent completions for the held-out evaluation questions.

To combat the capacity bottleneck of the initial run, the model was transitioned into a mid-data regime where the corpus was expanded to 1,200 training observations. This scaling brought a distinct quantitative shift, allowing 16 out of the 20 evaluation prompts to successfully cross the initial validation threshold of 80% or greater Hebrew character density. However, qualitative analysis exposed severe underlying alignment vulnerabilities. The model suffered from intense multi-lingual tokenization bleeding, frequently nesting raw English and Chinese subword characters inside Hebrew terms. Furthermore, severe conceptual drift and domain hallucinations were widespread. For instance, when prompted to describe how to brew a cup of tea, the model hallucinated an explicit guide for preparing coffee. Extensive hyperparameter searches yielded diminishing returns, failing to mitigate these hallucination vectors.

The stagnation in performance prompted a thorough data-quality audit of the synthetic corpus. The analysis revealed that the 1,200-sample dataset was highly contaminated with low-quality, ambiguous instructions and meta-prompts, such as “Ask the user what his instructions are.” These examples did not reflect genuine user distributions and lacked clear target answer bounds, introducing heavy gradient noise during optimization. Consequently, the data generation pipeline was completely overhauled to enforce rigid semantic quality controls, filtering out ambiguous prompts and maximizing domain uniqueness. This pipeline rectification resulted in a clean, highly curated, and vastly expanded training set of 5,000 high-quality instruction-response pairs, laying the groundwork for successful behavioral and conceptual alignment.

Second Experiments

The evaluation records and output files reveal a clear progression between two major fine-tuning configurations, mapping out how changes in the training architecture directly reshaped the model's linguistic behavior. The first experiment utilized the fully curated dataset of 5,000 instruction-response pairs but ran under a highly aggressive optimization window. The hyperparameters for this initial pass were configured with a training duration of 15 epochs. Due to hardware memory constraints on the single GPU runtime, the physical batch size was strictly capped at 4, which was paired with a gradient accumulation value of 1, resulting in an effective batch size of 4. The remaining parameters included a learning rate of $2e-4$, a LoRA rank of 16, a LoRA alpha of 32, and a LoRA dropout rate of 0.05. This run represents about 18.5 million trainable parameters out of the total 1.58 billion parameters. While this configuration successfully forced the network to adopt the Hebrew alphabet, achieving a script purity between 92% and 100%, the combination of extreme overtraining across 15 epochs and a tokenization masking offset caused a complete semantic breakdown. The model suffered from severe contextual hallucinations, completely detaching from the reality of the prompts while maintaining a confident grammatical flow. It substituted a cookie recipe when asked to describe an omelette, treated a data backup prompt as a tutorial on returning items to the shower, and explained the blue sky through a surreal fable about sunlight dancing around the moon.

The second experiment introduced structural changes to stabilize the model's focus and expand its parameter capacity while deploying a cosine learning rate scheduler to manage optimization step bounds across the training run. The updated hyperparameters were scaled down to 3 epochs to prevent overfitting. While the physical batch size remained capped at 4 due to the baseline hardware limitations, the virtual step size was reinforced by increasing the gradient accumulation steps to 4 to establish a stable effective batch size of 16. To provide the network with more expressive freedom to coordinate translations implicitly, the LoRA rank was expanded to 32 and the LoRA alpha to 64. The peak learning rate was kept at its threshold of $2e-$

4, decaying non-linearly according to the cosine annealing profile to force a fine-grained cooling phase toward zero in the final training cycles. Crucially, the dataset building script was corrected to tokenize the entire sequence as a single block, preventing the subtle token boundaries shift from misaligning the target labels. This final run represents about 37 million trainable parameters.

The output records from this final run demonstrate a complete resolution of the chaotic contextual hallucinations, showing that the model successfully retained its core world knowledge and accurately processed the underlying intent of the English queries. To prevent over-indexing on noisy token distributions and regularize the adaptation layer, a weight decay of 0.01 was applied to the optimizer. However, it developed a highly distinct symptom known as cross-lingual semantic interference. Instead of losing the topic entirely, the attention layers occasionally grabbed equivalent, highly concrete subword tokens from other languages present in its pre-training base to complete a sentence. This manifested as precise multilingual vocabulary injections embedded straight into fluent Hebrew syntax, alongside localized phonetic and narrative memory swaps. Quantitatively, this run represents a substantial structural alignment victory, with scores between 92% and 100% script purity.

Evaluation

The following sections present a detailed analysis of the 20 held-out evaluation instructions used to assess the model's cross-lingual capabilities. Each evaluation sample was initially vetted programmatically to verify that it met the baseline criteria of achieving at least an 80% Hebrew character density, as documented in the notes of the final eval_outputs.jsonl file. To provide a more granular understanding of model behavior, the automated metrics were paired with a comprehensive qualitative review combining human annotation and LLM-as-a-judge criteria, utilizing Gemini Flash 3.5. This joint evaluation grades each model response on two critical dimensions, factual correctness and structural coherence, using a standardized scale from 1 to 3, where 1 indicates an incorrect or failed output, 2 denotes a generally correct or partially aligned response, and 3 represents a completely correct, highly articulate execution.

Evaluation Sample 1: "Explain why the sky looks blue during the day."

Hebrew Density Metrics: Base Model: 0% | Finetuned Model: 96%

Human Annotator Review

The base model provides a highly accurate physical science explanation of Rayleigh scattering, detailing wavelength interactions and the shifted hue observed during sunset. The progression of ideas is completely logical and highly coherent. Conversely, the finetuned model suffers a total factual collapse, delivering an output that incorrectly claims the sky looks gray in the morning and evening due to strong sunlight, while mixing up compass directions with diurnal cycles.

- **Language Discrepancies:** The finetuned model exhibits severe semantic translation errors where grammatically correct Hebrew structures carry nonsensical vocabulary combinations. For example, it inaccurately claims the sun turns "Northeast or north" (Northeast לצפון) at noon, causing the light to appear dark-red.
- **Base Model Scores:** Factual Correctness: 3 | Structural Coherence: 3
- **Finetuned Model Scores:** Factual Correctness: 1 | Structural Coherence: 1

LLM-as-a-Judge Review

The base model achieves a flawless technical execution of the instruction in English. The finetuned model successfully attempts the target language shift to Hebrew, but its internal semantic logic breaks down completely, rendering the core explanation factually invalid.

- **Language Discrepancies:** The finetuned model meets character density goals but experiences severe cross-lingual alignment friction, leading to a breakdown in baseline factual sense during the translation process.
- **Base Model Scores:** Factual Correctness: 3 | Structural Coherence: 3
- **Finetuned Model Scores:** Factual Correctness: 1 | Structural Coherence: 2

Evaluation Sample 2: "Give two advantages and two disadvantages of public transportation."

Hebrew Density Metrics: Base Model: 0% | Finetuned Model: 93%

Human Annotator Review

The base model perfectly satisfies the explicit numeric constraints of the prompt, listing two distinct advantages and two clear disadvantages in well-structured English. The finetuned model transitions into Hebrew but fails the strict 2x2 formatting rule by blending points, while introducing an unnecessary English code-switch with the word "fortunes".

- **Language Discrepancies:** The finetuned model displays a script and vocabulary leak by improperly introducing the Latin-script English word "nfortunes" directly into the Hebrew sentence.
- **Base Model Scores:** Factual Correctness: 3 | Structural Coherence: 3
- **Finetuned Model Scores:** Factual Correctness: 2 | Structural Coherence: 2

LLM-as-a-Judge Review

The base model shows a highly structured, clean layout with appropriate paragraphing. The finetuned model demonstrates slightly halting sentence structures and includes a conceptual error by associating private vehicle acquisition expenses with public transit users.

- **Language Discrepancies:** The finetuned model suffers from cross-lingual script interference, allowing vocabulary from the source language to pollute the target language profile.
- **Base Model Scores:** Factual Correctness: 3 | Structural Coherence: 3
- **Finetuned Model Scores:** Factual Correctness: 2 | Structural Coherence: 2

Evaluation Sample 3: "Write a short email asking a professor for an extension on an assignment."

Hebrew Density Metrics: Base Model: 0% | Finetuned Model: 98%

Human Annotator Review

The base model outputs a highly professional, polite, and well-structured email template containing appropriate placeholders for student customization. The finetuned model fails to

maintain an acceptable academic register, adopting a tone that is far too casual for a professor, and experiences a total breakdown in structural coherence due to an unprompted injection of Chinese characters mid-sentence.

- **Language Discrepancies:** The finetuned model exhibits severe multi-token script corruption, randomly outputting a string of Chinese logographs ("我又") directly between Hebrew words.
- **Base Model Scores:** Factual Correctness: 3 | Structural Coherence: 3
- **Finetuned Model Scores:** Factual Correctness: 2 | Structural Coherence: 1

LLM-as-a-Judge Review

The base model fulfills the task by producing a standard, real-world professional email template. The finetuned model preserves the basic intent of the prompt within its Hebrew text, but severe token degradation causes multi-lingual corruption that completely breaks sentence formatting.

- **Language Discrepancies:** The finetuned model displays critical multi-lingual cross-contamination, proving that token vocabulary bounds degraded during the fine-tuning process to allow unrelated script leakage.
- **Base Model Scores:** Factual Correctness: 3 | Structural Coherence: 3
- **Finetuned Model Scores:** Factual Correctness: 2 | Structural Coherence: 1

Evaluation Sample 4: "Describe how to make a simple omelette."

Hebrew Density Metrics: Base Model: 0% | Finetuned Model: 100%

Human Annotator Review

The base model delivers a logical, standard, and easy-to-follow cooking guide in English. The finetuned model displays extreme hallucinatory behavior, employing bizarre translation terms like adding protein to a "crystal" shape, placing eggs into a "brush," and suggesting highly unconventional ingredients such as ice.

- **Language Discrepancies:** The finetuned model translates culinary concepts literalistically or via completely erroneous token mappings, leading to highly unnatural vocabulary selections in Hebrew such as using "crystal/crystalized shape" (גביש) and "brush" (מברשת) to describe an omelette pan and egg mixture.
- **Base Model Scores:** Factual Correctness: 3 | Structural Coherence: 3
- **Finetuned Model Scores:** Factual Correctness: 1 | Structural Coherence: 1

LLM-as-a-Judge Review

The base model establishes a clear, chronological sequence of steps. While the finetuned model mimics the grammatical structure of a standard recipe, its lexical choices are entirely nonsensical and fail basic culinary validation.

- **Language Discrepancies:** The finetuned model maps conceptual cooking actions to entirely wrong vocabulary items within the Hebrew dictionary, indicating a severe breakdown in cross-lingual semantic alignment.

- **Base Model Scores:** Factual Correctness: 3 | Structural Coherence: 3
- **Finetuned Model Scores:** Factual Correctness: 1 | Structural Coherence: 2

Evaluation Sample 5: "What is the difference between supervised and unsupervised learning?"

Hebrew Density Metrics: Base Model: 0% | Finetuned Model: 96%

Human Annotator Review

The base model provides excellent textbook definitions, accurately mapping the differences between labeled and unlabeled data alongside their respective machine learning applications. The finetuned model delivers incorrect technical translations for core computer science terminology and breaks vocabulary bounds by blending Hebrew, English, and Italian words.

- **Language Discrepancies:** The finetuned model mistranslates the technical term "supervised learning" to "final learning" (למידה סופית) and introduces severe multi-language lexical bleed, mixing the English word "where" and the Italian word "dove" directly into the text.
- **Base Model Scores:** Factual Correctness: 3 | Structural Coherence: 3
- **Finetuned Model Scores:** Factual Correctness: 1 | Structural Coherence: 1

LLM-as-a-Judge Review

The base model yields a high-quality, precise technical summary. The finetuned model exhibits a very loose conceptual understanding of the underlying paradigms, but this is severely undercut by poor vocabulary selection and language bleed.

- **Language Discrepancies:** The finetuned model suffers from language pollution across three distinct linguistic domains (Hebrew, English, Italian), demonstrating high tracking confusion during technical lookup tasks.
- **Base Model Scores:** Factual Correctness: 3 | Structural Coherence: 3
- **Finetuned Model Scores:** Factual Correctness: 1 | Structural Coherence: 2

Evaluation Sample 6: "Summarize the story of Cinderella in three sentences."

Hebrew Density Metrics: Base Model: 0% | Finetuned Model: 100%

Human Annotator Review

The base model provides a coherent narrative summary that respects the sentence length constraint, though it introduces a minor plot error regarding who breaks the glass slipper. The finetuned model experiences a major crossover hallucination by introducing Geppetto from Pinocchio into the plot, though it does manage to output exactly three sentences as requested.

- **Language Discrepancies:** While the finetuned model successfully outputs the summary in fluent Hebrew grammar, its cross-lingual knowledge retrieval maps the narrative characters to the wrong cultural entity, inserting Geppetto (ג'פֶּטוֹ) into a Cinderella summary.
- **Base Model Scores:** Factual Correctness: 2 | Structural Coherence: 3

- **Finetuned Model Scores:** Factual Correctness: 1 | Structural Coherence: 2

LLM-as-a-Judge Review

The base model structures a well-formed text with an acceptable flow despite its small plot error. The finetuned model perfectly respects the quantitative constraints of the prompt, but fails core factual checks completely.

- **Language Discrepancies:** The finetuned model satisfies cross-lingual target density goals perfectly and shows strong syntactic flow, but its internal semantic token dependencies fail by crossing over separate fairy tale records.
- **Base Model Scores:** Factual Correctness: 2 | Structural Coherence: 3
- **Finetuned Model Scores:** Factual Correctness: 1 | Structural Coherence: 3

Evaluation Sample 7: "Suggest three ways to reduce smartphone distraction while studying."

Hebrew Density Metrics: Base Model: 0% | Finetuned Model: 100%

Human Annotator Review

The base model lists solid, practical tips, though its third recommendation leans slightly toward general environment design rather than phone-specific adjustments. The finetuned model provides two actionable points in Hebrew but terminates with an unnatural and confusing colloquial idiom involving a refrigerator.

- **Language Discrepancies:** The finetuned model exhibits an idiomatic discrepancy in Hebrew by translating or generating a metaphor ("to turn the table into work and not a refrigerator" / ולא למקרר) that completely lacks functional meaning in the context of study habits.
- **Base Model Scores:** Factual Correctness: 3 | Structural Coherence: 3
- **Finetuned Model Scores:** Factual Correctness: 2 | Structural Coherence: 2

LLM-as-a-Judge Review

The base model utilizes a clear, actionable layout. The finetuned model scores high on structural layout and target language alignment, displaying only minor logical oddities at the close of its response.

- **Language Discrepancies:** The finetuned model achieves clean Hebrew character density, but suffers a minor linguistic phrasing mismatch due to an ungrounded figurative expression at the end of the text.
- **Base Model Scores:** Factual Correctness: 3 | Structural Coherence: 3
- **Finetuned Model Scores:** Factual Correctness: 2 | Structural Coherence: 3

Evaluation Sample 8: "Explain what happens when water boils."

Hebrew Density Metrics: Base Model: 0% | Finetuned Model: 100%

Human Annotator Review

The base model presents a scientifically accurate explanation of the liquid-to-gas transition, kinetic energy, and environmental variables. The finetuned model delivers completely inverted physics and chemistry, claiming that when water cools down, it creates a gaseous form called carbon dioxide.

- **Language Discrepancies:** The finetuned model suffers from severe semantic vocabulary inversion in Hebrew, completely swapping the term for heating with "cools down" (מתקרר), and substituting "carbon dioxide" (פחמן דו־חמצני) instead of water vapor or steam.
- **Base Model Scores:** Factual Correctness: 3 | Structural Coherence: 3
- **Finetuned Model Scores:** Factual Correctness: 1 | Structural Coherence: 2

LLM-as-a-Judge Review

The base model shows a solid, baseline presentation of elementary physics. The finetuned model produces syntactically fluent sentence structures, but the statements themselves are entirely counter-factual.

- **Language Discrepancies:** The finetuned model shows excellent Hebrew grammar and syntax, but its vocabulary tokens map onto deeply flawed conceptual definitions, exposing a disconnect between language generation and facts.
- **Base Model Scores:** Factual Correctness: 3 | Structural Coherence: 3
- **Finetuned Model Scores:** Factual Correctness: 1 | Structural Coherence: 2

Evaluation Sample 9: "Give a polite refusal to an invitation to a party."

Hebrew Density Metrics: Base Model: 0% | Finetuned Model: 92%

Human Annotator Review

The base model writes a standard, courteous, and highly appropriate refusal message in English. The finetuned model crafts a polite response with an excellent emotional tone, but slightly drops in readability due to inserting the Romanian word for invitation alongside the Hebrew equivalent.

- **Language Discrepancies:** The finetuned model experiences vocabulary leakage, introducing an unexpected Romance language token ("invitație") directly into an otherwise fluent Hebrew paragraph.
- **Base Model Scores:** Factual Correctness: 3 | Structural Coherence: 3
- **Finetuned Model Scores:** Factual Correctness: 3 | Structural Coherence: 2

LLM-as-a-Judge Review

The base model matches real-world social expectations. The finetuned model successfully completes the underlying cross-lingual goal of the task despite experiencing minor vocabulary leakage.

- **Language Discrepancies:** The finetuned model shows high cross-lingual target compliance, but the output is slightly marred by multi-lingual token bleeding where Romanian merges with the Hebrew text.
- **Base Model Scores:** Factual Correctness: 3 | Structural Coherence: 3
- **Finetuned Model Scores:** Factual Correctness: 3 | Structural Coherence: 2

Evaluation Sample 10: "Turn the idea "mistakes are part of learning" into encouragement for a child."

Hebrew Density Metrics: Base Model: 0% | Finetuned Model: 95%

Human Annotator Review

The base model generates a concise, accurate motivational quote. The finetuned model encapsulates the core advice well within bolded text, but severely harms overall readability by outputting an internal monologue that mixes Hebrew, English, and Chinese scripts.

- **Language Discrepancies:** The finetuned model displays severe multi-script contamination during meta-instructions, outputting the English word "Advice" and the Chinese logographs "學生" (student) directly inside its introductory Hebrew sentence.
- **Base Model Scores:** Factual Correctness: 3 | Structural Coherence: 3
- **Finetuned Model Scores:** Factual Correctness: 2 | Structural Coherence: 1

LLM-as-a-Judge Review

The base model hits the precise requirements of the instruction. The finetuned model provides a structurally sound advice sentence, but the surrounding introductory syntax demonstrates multi-token language leakage.

- **Language Discrepancies:** The finetuned model displays structural token bleeding across three entirely different language families (Hebrew, English, and Chinese), highlighting severe cross-lingual alignment friction during formatting.
- **Base Model Scores:** Factual Correctness: 3 | Structural Coherence: 3
- **Finetuned Model Scores:** Factual Correctness: 2 | Structural Coherence: 2

Evaluation Sample 11: "Explain in simple terms how a rainbow forms."

Hebrew Density Metrics: Base Model: 0% | Finetuned Model: 94%

Human Annotator Review

The base model provides a simple, accurate explanation covering light refraction, reflection, and the splitting of light within droplets. The finetuned model successfully mirrors this accurate physical reasoning in Hebrew, but slips an English word into its concluding sentence.

- **Language Discrepancies:** The finetuned model achieves strong Hebrew output but exhibits a minor vocabulary leak at the very end of the text, writing the English word "arranged" inside a Hebrew sentence.
- **Base Model Scores:** Factual Correctness: 3 | Structural Coherence: 3

- **Finetuned Model Scores:** Factual Correctness: 3 | Structural Coherence: 2

LLM-as-a-Judge Review

The base model delivers a highly coherent and standard response. The finetuned model earns high marks for preserving scientific correctness while maintaining robust compliance with the targeted language profile.

- **Language Discrepancies:** The finetuned model exhibits high linguistic fidelity, showing only a single minor multi-lingual code-switch into English at its conclusion.
- **Base Model Scores:** Factual Correctness: 3 | Structural Coherence: 3
- **Finetuned Model Scores:** Factual Correctness: 3 | Structural Coherence: 3

Evaluation Sample 12: "List three tips for staying focused while working from home."

Hebrew Density Metrics: Base Model: 0% | Finetuned Model: 100%

Human Annotator Review

The base model generates highly relevant, professional recommendations with comprehensive sub-explanations. The finetuned model scores perfectly on language density, offering practical steps, though its second tip features a slightly awkward vocabulary choice regarding a work tray.

- **Language Discrepancies:** The finetuned model achieves 100% Hebrew density, but translates organizational habits using a slightly rigid and unnatural lexical selection ("work tray" / מגש עבודה) to describe a workstation setup.
- **Base Model Scores:** Factual Correctness: 3 | Structural Coherence: 3
- **Finetuned Model Scores:** Factual Correctness: 3 | Structural Coherence: 3

LLM-as-a-Judge Review

The base model shows exceptional depth and high formatting quality. The finetuned model achieves an ideal numerical list generation that aligns perfectly with cross-lingual instruction constraints.

- **Language Discrepancies:** The finetuned model demonstrates flawless target language retention without any token or script leakage, completely containing its response within Hebrew characters.
- **Base Model Scores:** Factual Correctness: 3 | Structural Coherence: 3
- **Finetuned Model Scores:** Factual Correctness: 3 | Structural Coherence: 3

Evaluation Sample 13: "Write a two-sentence thank-you note to a friend who helped you move."

Hebrew Density Metrics: Base Model: 0% | Finetuned Model: 100%

Human Annotator Review

The base model produces friendly text but fails the strict length constraint by structuring the note like an email template with a formal signature block. The finetuned model achieves an

exceptional result, executing exactly two sentences of natural, grammatically flawless Hebrew gratitude.

- **Language Discrepancies:** The finetuned model displays perfect mastery of native Hebrew phrasing, capturing tone, semantics, and script flawlessly without any cross-lingual anomalies or odd word choices.
- **Base Model Scores:** Factual Correctness: 2 | Structural Coherence: 3
- **Finetuned Model Scores:** Factual Correctness: 3 | Structural Coherence: 3

LLM-as-a-Judge Review

The base model falls short on length-based constraints. The finetuned model delivers a flawless performance, outperforming the baseline on specific structural rules.

- **Language Discrepancies:** The finetuned model represents the highest tier of cross-lingual fine-tuning success, aligning structural rules with completely natural and uniform target language generation.
- **Base Model Scores:** Factual Correctness: 2 | Structural Coherence: 3
- **Finetuned Model Scores:** Factual Correctness: 3 | Structural Coherence: 3

Evaluation Sample 14: "What is the difference between weather and climate?"

Hebrew Density Metrics: Base Model: 0% | Finetuned Model: 97%

Human Annotator Review

The base model gives a concise, accurate short-term versus long-term environmental distinction. The finetuned model retains a basic conceptual understanding but suffers significantly from multi-script character bleed (Chinese) and an explicit semantic word typo that swaps decades for smoke.

- **Language Discrepancies:** The finetuned model suffers from multi-script contamination, inserting a string of Chinese logographs ("长时间的模式") mid-sentence, and encounters a phonetic/semantic slip in Hebrew by typing "smoke" (עשן) instead of "decade" (עשור).
- **Base Model Scores:** Factual Correctness: 3 | Structural Coherence: 3
- **Finetuned Model Scores:** Factual Correctness: 2 | Structural Coherence: 1

LLM-as-a-Judge Review

The base model delivers a perfect factual summary. The finetuned model achieves acceptable raw language alignment scores but yields a severely degraded final presentation due to foreign character leakage.

- **Language Discrepancies:** The finetuned model fulfills bulk automated language metrics but fails qualitative review due to severe script pollution and erratic token selection that degrades the readability of the text.
- **Base Model Scores:** Factual Correctness: 3 | Structural Coherence: 3

- **Finetuned Model Scores:** Factual Correctness: 2 | Structural Coherence: 2

Evaluation Sample 15: "Describe the steps to brew a cup of tea."

Hebrew Density Metrics: Base Model: 0% | Finetuned Model: 98%

Human Annotator Review

The base model writes a highly descriptive, accurate seven-step guide. The finetuned model suffers an absolute recipe hallucination, directing the user in Hebrew to melt butter or cocoa in a cup, which describes hot chocolate or baking rather than tea preparation.

- **Language Discrepancies:** The finetuned model writes in grammatically sound Hebrew, but its cross-lingual concept association breaks down completely, mapping the instruction for "tea steps" to baking vocabulary ("butter", "cocoa" / חמאה או קקאו) within its semantic database.
- **Base Model Scores:** Factual Correctness: 3 | Structural Coherence: 3
- **Finetuned Model Scores:** Factual Correctness: 1 | Structural Coherence: 2

LLM-as-a-Judge Review

The base model establishes a logical sequential breakdown. The finetuned model successfully formats its response as a step-by-step instructional protocol, but the text itself belongs to a completely different query.

- **Language Discrepancies:** The finetuned model maintains structural target language fidelity but displays complete conceptual drift, outputting perfect Hebrew syntax linked to completely irrelevant culinary content.
- **Base Model Scores:** Factual Correctness: 3 | Structural Coherence: 3
- **Finetuned Model Scores:** Factual Correctness: 1 | Structural Coherence: 2

Evaluation Sample 16: "Give one reason to learn a second language and one reason it can be hard."

Hebrew Density Metrics: Base Model: 0% | Finetuned Model: 100%

Human Annotator Review

The base model details highly accurate cognitive benefits and structural language challenges. The finetuned model successfully mirrors these accurate ideas in natural Hebrew, experiencing only a minor stylistic repetition when describing sentence difficulties.

- **Language Discrepancies:** The finetuned model shows seamless cross-lingual translation capabilities, delivering an output with natural phrasing, appropriate vocabulary depth, and correct technical alignment in Hebrew.
- **Base Model Scores:** Factual Correctness: 3 | Structural Coherence: 3
- **Finetuned Model Scores:** Factual Correctness: 3 | Structural Coherence: 3

LLM-as-a-Judge Review

The base model demonstrates excellent coherence and depth. The finetuned model perfectly satisfies all explicit constraints, delivering precisely one advantage and one disadvantage in the target language.

- **Language Discrepancies:** The finetuned model successfully decouples from the source prompt language, achieving zero token leakage and complete target language adherence.
- **Base Model Scores:** Factual Correctness: 3 | Structural Coherence: 3
- **Finetuned Model Scores:** Factual Correctness: 3 | Structural Coherence: 3

Evaluation Sample 17: "Summarize what photosynthesis does for a plant."

Hebrew Density Metrics: Base Model: 0% | Finetuned Model: 100%

Human Annotator Review

The base model yields a precise, biology-focused academic breakdown of chemical stages. The finetuned model provides a successful high-level overview in Hebrew, making only a minor semantic slip by stating that a plant warms up carbon dioxide instead of absorbing it.

- **Language Discrepancies:** The finetuned model handles dense biochemical concepts well in Hebrew, though it falls into a minor linguistic discrepancy by selecting an incorrect action verb, stating that the plant "warms up carbon dioxide" (מתחממת פחמן) (דו-חמצני).
- **Base Model Scores:** Factual Correctness: 3 | Structural Coherence: 3
- **Finetuned Model Scores:** Factual Correctness: 2 | Structural Coherence: 3

LLM-as-a-Judge Review

The base model outlines an exceptionally detailed scientific synthesis. The finetuned model receives high marks for delivering a simple, accessible summary that remains entirely legible and fluent.

- **Language Discrepancies:** The finetuned model demonstrates robust target language conversion, outputting a clear, highly coherent explanation completely within expected Hebrew lexical boundaries.
- **Base Model Scores:** Factual Correctness: 3 | Structural Coherence: 3
- **Finetuned Model Scores:** Factual Correctness: 3 | Structural Coherence: 3

Evaluation Sample 18: "Suggest a simple plan to start running for a complete beginner."

Hebrew Density Metrics: Base Model: 0% | Finetuned Model: 99%

Human Annotator Review

The base model provides a practical, comprehensive weekly training calendar. The finetuned model introduces bizarre advice instructing a beginner to carry a comfortable umbrella to a solid location, and further breaks down by injecting a comparative Chinese word mid-sentence.

- **Language Discrepancies:** The finetuned model suffers from cross-lingual script contamination, randomly drawing a Chinese comparative token ("עדיף不如") into the middle of a Hebrew phrase and fracturing the target language profile. It also hallucinated an odd phrase to take an "umbrella" (מטרייה).
- **Base Model Scores:** Factual Correctness: 3 | Structural Coherence: 3
- **Finetuned Model Scores:** Factual Correctness: 2 | Structural Coherence: 1

LLM-as-a-Judge Review

The base model demonstrates an exceptional scheduling layout. The finetuned model relies on sound habit-building logic, but its linguistic clarity is heavily degraded by multi-token character leakage.

- **Language Discrepancies:** The finetuned model introduces tokens from an unrelated foreign character vocabulary, showing an absolute breakdown in language boundary enforcement mid-sentence.
- **Base Model Scores:** Factual Correctness: 3 | Structural Coherence: 3
- **Finetuned Model Scores:** Factual Correctness: 2 | Structural Coherence: 2

Evaluation Sample 19: "Explain why we should back up important files."

Hebrew Density Metrics: Base Model: 0% | Finetuned Model: 100%

Human Annotator Review

The base model displays an exhaustive, multi-faceted overview of data security and business continuity. The finetuned model generates a highly concise and correct two-sentence rationale regarding system recovery, though it utilizes an unconventional phrasing choice by referencing information injury rather than damage.

- **Language Discrepancies:** The finetuned model effectively operates in Hebrew, though its technical translation choices lean on an unusual literal application of a physical term ("injury to the information" / לפציעה של המידע) instead of standard digital data corruption or damage terminology.
- **Base Model Scores:** Factual Correctness: 3 | Structural Coherence: 3
- **Finetuned Model Scores:** Factual Correctness: 3 | Structural Coherence: 3

LLM-as-a-Judge Review

The base model exhibits thorough, analytical formatting. The finetuned model delivers an excellent response that succinctly and accurately captures the fundamental rationale for data preservation.

- **Language Discrepancies:** The finetuned model preserves full target language execution, ensuring the output matches the required cross-lingual direction constraints perfectly with zero script anomalies.
- **Base Model Scores:** Factual Correctness: 3 | Structural Coherence: 3
- **Finetuned Model Scores:** Factual Correctness: 3 | Structural Coherence: 3

Evaluation Sample 20: "Turn the idea “mistakes are part of learning” into encouragement for a child."

Hebrew Density Metrics: Base Model: 0% | Finetuned Model: 95%

Human Annotator Review

The base model outputs a warm, highly encouraging, and age-appropriate paragraph. The finetuned model matches this high standard by delivering an equally warm, coherent, and motivating message entirely in natural Hebrew, correctly reframing errors as small warnings rather than failures.

- **Language Discrepancies:** The finetuned model achieves total linguistic fluency in Hebrew, seamlessly adjusting its emotional tone and sentence structure without any vocabulary discrepancies or formatting errors.
- **Base Model Scores:** Factual Correctness: 3 | Structural Coherence: 3
- **Finetuned Model Scores:** Factual Correctness: 3 | Structural Coherence: 3

LLM-as-a-Judge Review

The base model shows strong text alignment and emotional resonance. The finetuned model exhibits flawless cross-lingual behavioral integration, avoiding tracking errors or cross-script bleeding entirely.

- **Language Discrepancies:** The finetuned model demonstrates pristine target language formatting, achieving the ideal balance of factual retention and cross-lingual output compliance without tracking errors.
- **Base Model Scores:** Factual Correctness: 3 | Structural Coherence: 3
- **Finetuned Model Scores:** Factual Correctness: 3 | Structural Coherence: 3

Overall Cross-Lingual Fine-Tuning Results

The LoRA fine-tuning of the model successfully met its primary linguistic target by consistently forcing responses to be generated in Hebrew when prompted with English instructions. Across all 20 evaluation instructions, the fine-tuned model consistently cleared the automated threshold of achieving at least an 80% Hebrew character density, frequently reaching near-total or complete language compliance. However, forcing this cross-lingual shift through low-rank adaptation introduced a severe "alignment tax" that compromised both factual integrity and structural coherence, resulting in erratic performance that fluctuated heavily depending on the domain of the prompt.

Linguistic analysis highlights a pronounced tracking vulnerability in the fine-tuned model's token generation layer, frequently resulting in multilingual vocabulary leaks and script contamination. The model repeatedly injected tokens from completely unrelated languages, blending English ("fortunes"), Romanian ("invitație"), Italian ("dove"), and Chinese logographs ("我又") directly into its Hebrew sentences.

Crucially, this cross-lingual friction caused severe cognitive degradation and factual drift in high-precision or technical contexts. The model routinely fell victim to hallucinations, which manifested in four distinct ways:

- **Technical Inversions:** It mistranslated foundational machine learning terms into nonsensical literal concepts, such as mapping "supervised learning" to "final learning".
- **Scientific Failures:** It inverted core physical science rules, explicitly claiming that boiling water is a cooling process that generates carbon dioxide gas.
- **Procedural Drift:** It mapped step-by-step instructions onto completely wrong domains, entirely substituting a tea-brewing prompt with a recipe involving cocoa and butter.
- **Narrative Crossovers:** It suffered database lookup mix-ups during storytelling tasks, bizarrely injecting Geppetto into a three-sentence summary of Cinderella.

Conversely, the fine-tuned model excelled when executing short, conversational, and emotionally grounded tasks. In samples requiring brief social formatting, such as writing a two-sentence thank-you note or transforming a lesson into encouragement for a child, the model achieved pristine, grammatically flawless Hebrew phrasing that was completely free of token leakage or factual errors. These findings indicate that while LoRA can rapidly re-index the linguistic generation layer, it compromises internal token dependencies in complex reasoning tracks, highlighting the need for more robust data filtering or regularization to stabilize deeper semantic structures.